

```
In [ ]: from pathlib import Path

import marimo as mo
ASSETS = mo.notebook_dir() / "assets"
DEMO_URL = Path("https://llm-demos.streamlit.app/")
```

LLM deep dive: what's in there?



Goals

- Demystify jargon
- Demo foundational concepts
- Strengthen mental models around LLMs

What we won't cover

Important topics that are out of scope:

- Ethical concerns
- Ecosystems surrounding the models (harnesses, agents)
- Non-language examples (music, image generation)
- Practical use cases

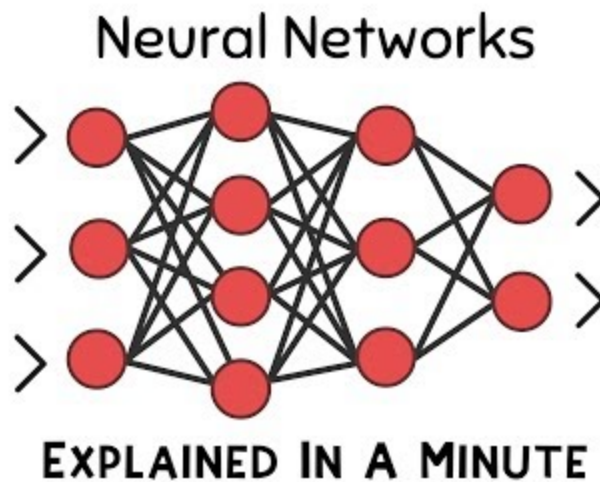
Large Language Models (LLMs)

A language model of sufficient size and complexity that interprets and generates text using natural human language. LLMs can predict text from input, based on their training from large amounts of text data.

- LLMs are neural networks
- Usually refers to models based on a transformer architecture
- *Attention is All You Need*: 2017 paper that started our current era of language models

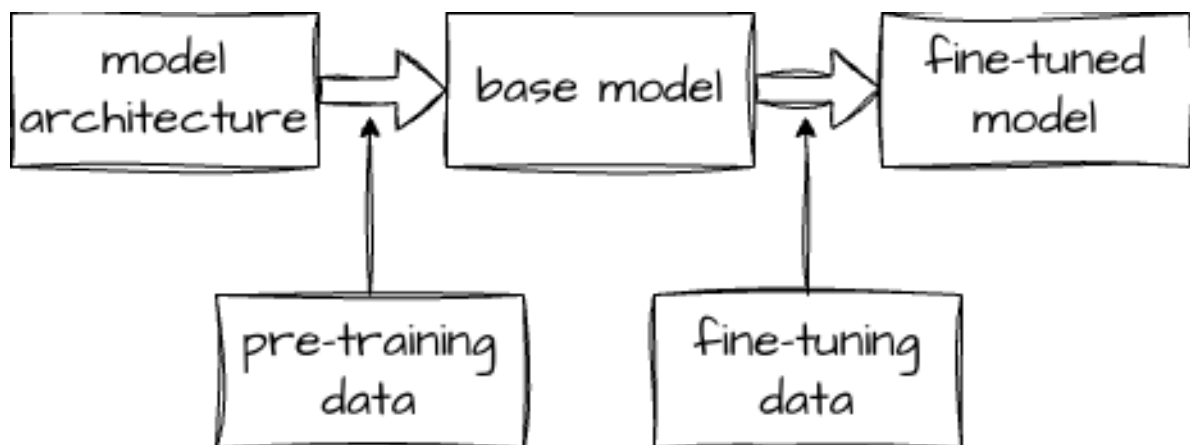
Yes, but what is a neural network?

Click below for a one-minute explainer



Creating an LLM's neural network

- architecture: defines a model's vocabulary, number of network layers, and other parameters
- training process: creates the embeddings and weights used when predicting words



It all starts with tokens

- lego pieces that represent the most atomic units of language
- represented by integers called *token ids*
- every model has its own "tokenizer" that determines how language is broken down into tokens
- a model's *vocabulary* is the number of unique tokens a model understands and generates

Demo: tokenization

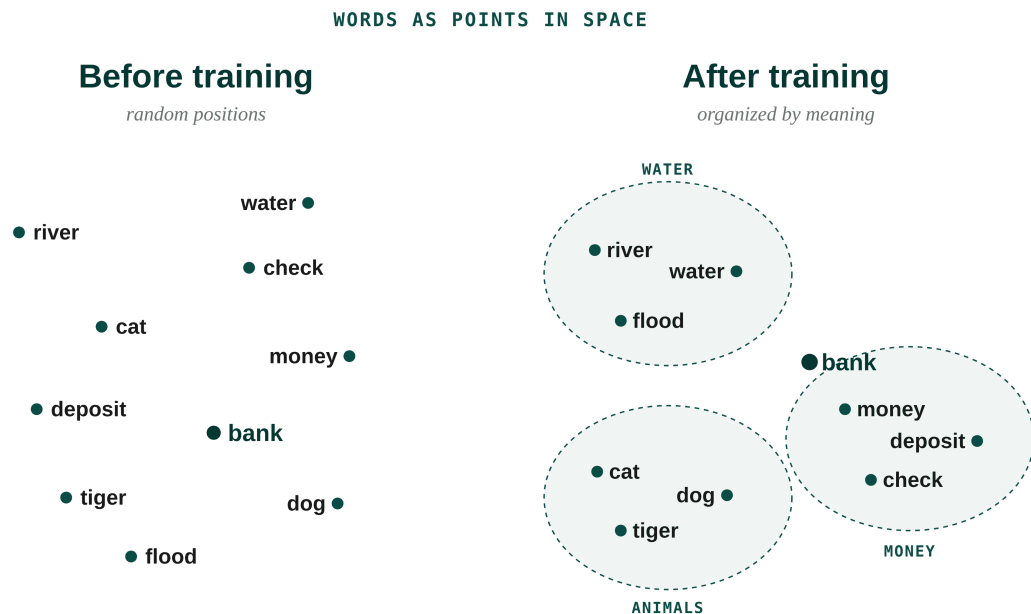
<https://llm-demos.streamlit.app/tokenizer>

Jargon: embeddings

- Each token has an embedding that captures how it relates to every other token in the vocabulary
- An *embedding* is a list of numbers
- The training process adjusts the embeddings until similar tokens end up with similar values

```
In [ ]: mo.vstack([
    mo.md("""
# Embeddings: _bank_
"""),
    mo.md("&nbsp;"),
    mo.image(src=ASSETS / "embedding_space_before_after_training.png",
width=700),
])
```

Embeddings: *bank*



```
In [ ]: mo.vstack([
    mo.md("""
# Embeddings: model's view
"""),
    mo.md("&nbsp;"),
    mo.image(src=ASSETS / "embeddings_as_model_data.png", width=700),
])
```

Embeddings: model's view

WHAT THE MODEL SEES

Before training

random numbers

river	[	0.42	-0.81	0.17	-0.05	0.66	]
water	[	-0.23	0.54	-0.91	0.38	-0.12	]
flood	[	0.71	0.09	-0.34	-0.62	0.28	]
cat	[	-0.55	-0.18	0.83	0.41	-0.07	]
tiger	[	0.14	-0.47	0.05	0.79	-0.88	]
dog	[	0.68	0.31	-0.22	-0.94	0.51	]
bank	[	-0.36	0.72	0.44	-0.19	-0.65	]
money	[	0.27	-0.93	0.61	0.08	0.39	]
deposit	[	-0.04	0.46	-0.77	0.52	-0.31	]
check	[	0.89	-0.15	0.33	-0.58	0.02	]

After training

patterns by meaning

river	[	0.81	0.74	0.69	-0.12	0.05	]
water	[	0.79	0.71	0.72	-0.09	0.08	]
flood	[	0.77	0.68	0.65	-0.15	0.11	]
cat	[	-0.08	0.11	-0.06	0.82	0.76	]
tiger	[	-0.04	0.09	-0.11	0.78	0.81	]
dog	[	-0.07	0.14	-0.08	0.85	0.73	]
bank	[	0.41	0.38	0.35	0.42	0.39	]
money	[	-0.05	0.08	-0.11	0.79	0.74	]
deposit	[	-0.09	0.12	-0.08	0.81	0.77	]
check	[	-0.06	0.09	-0.10	0.83	0.71	]

■ water-related dimensions

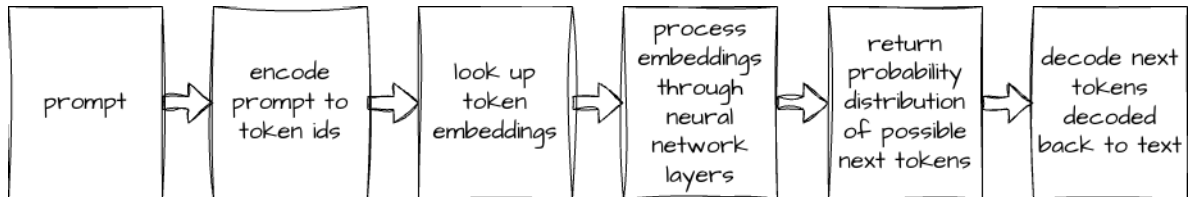
■ animal-related dimensions

■ money-related dimensions

```
In [ ]: mo.vstack([
    mo.md("""
    # Jargon – inference
    – _Inference_ is what happens when you send a prompt to an LLM
    – The last token in the prompt represents the token _and_ everything
    the model has "figured out" about the rest of the prompt
    """),
    mo.md("&nbsp;"),
    mo.image(src=ASSETS / "llm_inference.png"),
])
```

Jargon - inference

- *Inference* is what happens when you send a prompt to an LLM
- The last token in the prompt represents the token *and* everything the model has "figured out" about the rest of the prompt



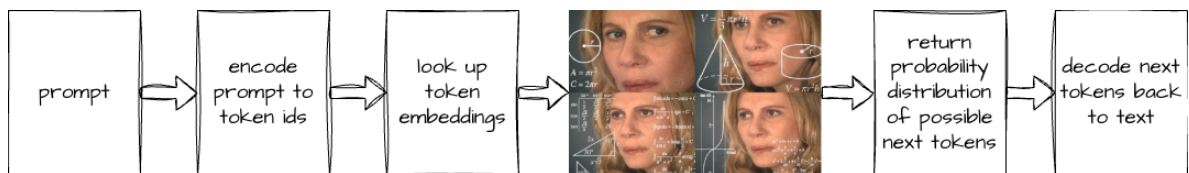
```
In [ ]: mo.md(f"""  
# Demo: inference  
[{{DEMO_URL}}/inference]({{DEMO_URL}}/next_token)  
""")
```

Demo: inference

<https://llm-demos.streamlit.app/inference>

```
In [ ]: mo.vstack([  
    mo.md("""  
# "Embeddings through neural network layers" is not auditable  
"""),  
    mo.md("&nbsp;"),  
    mo.image(src=ASSETS / "llm_inference_wat.png"),  
)
```

"Embeddings through neural network layers" is not auditable



Discussion and last thoughts

- Biases

- Some organizations release "open weights" models (Google's Gemma family)
- Typically, these "open weights" models don't publish their training data
- Retaining voice

```
In [ ]: def main() -> None:
import subprocess
import sys
from pathlib import Path

notebook = Path(__file__).resolve()
marimo_bin = Path(sys.executable).with_name("marimo")
raise SystemExit(
    subprocess.call([str(marimo_bin), "run", str(notebook),
*sys.argv[1:]])
)
```